20 AUGUST 2026

Coding Café

Environments & first programs

Outline

1. What is a programming language?
2. Characterising a language
3. Python
4. Some programming jargon
5. Setting up your machine
6. Your first programs



Natural Language

- “Calculate the total amount of the bill, how much do I owe you?”
- “Go left if the intersection is blocked.”
- “If the assignment is late, no marks will be given.”



Machine Language

- Data transfer (MOV, LDR, STR)
- Arithmetic (ADD, SUB, MUL)
- Logical (AND, ORR, EOR)
- Comparison (CMP, TST)
- Program branching (B, BL, CBZ)

ARM64 — the instruction set in your phone



Natural Language

- “Calculate the total amount of the bill, how much do I owe you?”
- “Go left if the intersection is blocked.”
- “If the assignment is late, no marks will be given.”

Outline

1. What is a programming language?
2. Characterising a language
3. Python
4. Some programming jargon
5. Setting up your machine
6. Your first programs

CHARACTERISING A LANGUAGE

Syntax

GRAMMAR OF THE LANGUAGE

Semantics

MEANING CONVEYED BY THE LANGUAGE

CHARACTERISING A LANGUAGE

Syntax

GRAMMAR OF THE LANGUAGE

Valid arrangements of the parts of a language.

English: Subject + verb + object

She sat on the chair

Python: Keyword/definition + condition + action

```
if (age > 25):
    print("You may enter the bar")
```

Symbols + **Rules**

Semantics

MEANING CONVEYED BY THE LANGUAGE

Syntax

GRAMMAR OF THE LANGUAGE

Valid arrangements of the parts of a language.

English: Subject + verb + object

She sat on the chair

Python: Keyword/definition + condition + action

```
if (age > 25):
    print("You may enter the bar")
```

Symbols + **Rules**

Semantics

MEANING CONVEYED BY THE LANGUAGE

1. Static Semantics:
   - More nuanced rules; implicit
   - Violation of these confounds meaning
   - *numbers can only talk to other numbers*

2. Dynamic Semantics:
   - Each statement should be meaningful and offer a single good meaning; precise

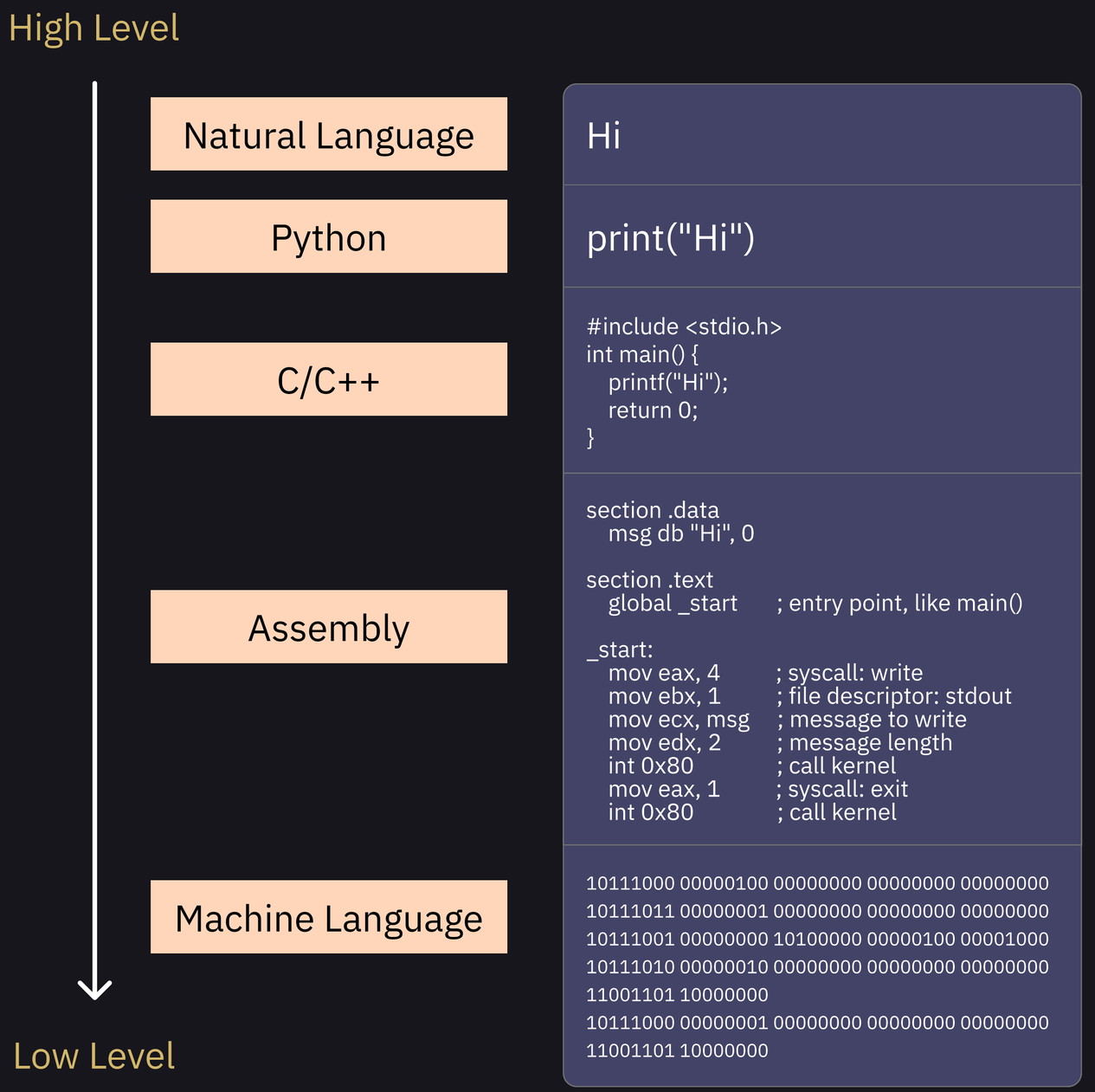
Meaning

CHARACTERISING A LANGUAGE

If natural languages are good for humans and machine languages are good for computers, what do you think makes a good programming language?

Should it be more like human language or more like machine language?

Depending on where a programming language sits, the balance of pros and cons shifts in different directions.



Outline

1. What is a programming language?
2. Characterising a language
3. Python
4. Some programming jargon
5. Setting up your machine
6. Your first programs

1. High-level language
    - developer friendly
2. Dynamically typed
    - type decided at runtime
    - no need to specify data type
3. Interpreted
    - line by line, not of all together
    - faster to develop and test

PYTHON

1. High-level language
   - developer friendly
2. Dynamically typed
   - type decided at runtime
   - no need to specify data type
3. Interpreted
   - line by line, not of all together
   - faster to develop and test

C

```c
int checkPalindrome(char word[]) {
    int length, i, j;
        length = strlen(word);
        i = 0;
        j = length - 1;

    while (i <= j)
        if (word[i] == word[j]) {
            i = i + 1;
            j = j - 1;
        }
        else break;

    if (i > length / 2)
        return 1;
    else
        return 0;
}
```

Python

```python
def checkPalindrome(word):
    if (word == word[::-1]):
        return True;
    else:
        return False;
```

PYTHON

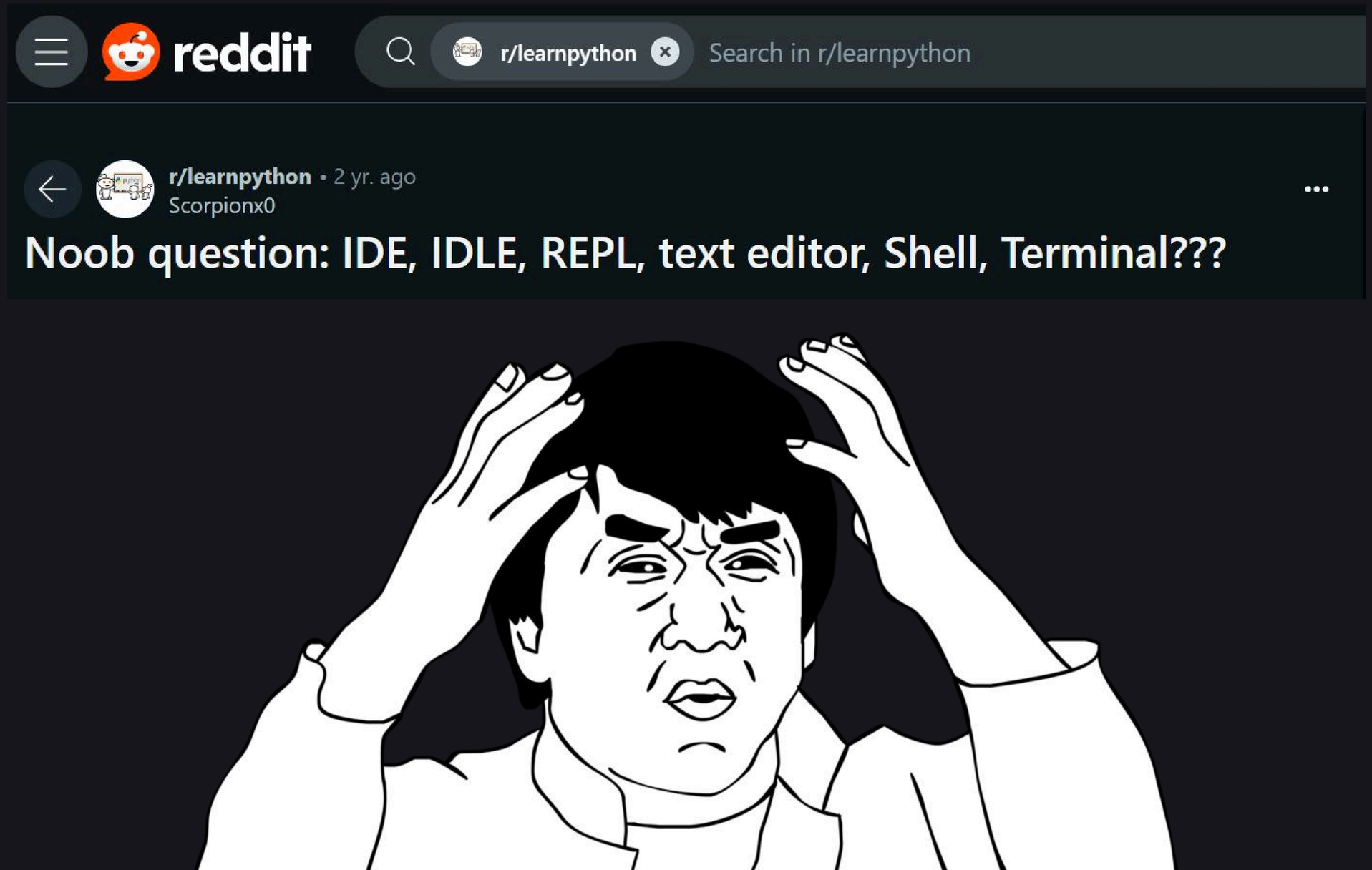
1. High-level language
2. Dynamically typed
3. Interpreted
4. Widely supported
5. Easy to learn

Web frameworks	django, flask
Mathematical libraries	math, numpy
Scientific packages	pandas, scipy
ML/AI libraries	pytorch, tensorflow
Data visualisation	tkinter, matplotlib
Gamedev frameworks	pygame, pyglet

Outline

1. What is a programming language?
2. Characterising a language
3. Python
4. Some programming jargon
5. Setting up your machine
6. Your first programs

SOME PROGRAMMING JARGON



SOME PROGRAMMING JARGON

IDE








Integrated Development Environment

IDLE



Integrated Development
Learning Environment

REPL



Read, Evaluate, Print, Loop

Shell





The interpreter between you
and the computer

Terminal



Graphical home of the shell

Text Editor



Just a text editor...that's it

Outline

1. What is a programming language?
2. Characterising a language
3. Python
4. Some programming jargon
5. Setting up your machine
6. Your first programs

1. Get Python from python.org/downloads
2. Install the latest 3.14 release
3. Windows: tick “Add python.exe to PATH”
4. macOS and Linux: you may already have one — check before installing
5. Verify the installation →

In the terminal, each line will begin with something like ~$ or C:\> depending on your OS. This is known as the prompt and it tells you location where you are on your machine.

You type your instructions here.

After installation, verify it in the terminal

macOS/linux terminal

```
# ON Linux / WSL / macOS

~$ python3 --version
Python 3.14.6 ← if you see this you’re good
```

windows cmd

```
# ON Windows CMD

C:\> python --version
Python 3.14.6 ← if you see this you’re good

C:\> py --version
Python 3.14.6 ← if you see this you’re good
```

1. Get VS Code from code.visualstudio.com
2. Install the Python extension by Microsoft
3. Open a folder, not a file — VS Code works in folders
4. Pick your interpreter: Cmd/Ctrl + Shift + P → “Python: Select Interpreter”*

You can also open it from the terminal

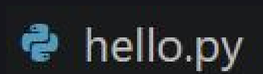
terminal

```
# From inside the folder open a terminal

~$ code .
```

* This will anchor the Python version to be used for the current project. Without this it will default back to the global installation. **Both work**.

YOUR FIRST .PY FILE

1. Make one folder for this course and keep everything in it
2. New file → hello.py (on renaming, it should show the Python logo next to the file,  if it doesn't, you have created a folder)
3. The .py extension is what makes it a Python file
4. Save, then run it — from VS Code (play button), or from the terminal →

```python
# hello.py
print("Hello, Coding Cafe")
```

terminal

```
# In the same location as your .py file

~$ python3 hello.py
Hello, Coding Cafe
```

Outline

1. What is a programming language?
2. Characterising a language
3. Python
4. Some programming jargon
5. Setting up your machine
6. Your first programs

Two functions to ask and tell: `input()` and `print()`

Accepting input from the user - `input("Some prompt to the user here")`

```
>>> customer_reply = input("What would like help with today?: ")
What would like help with today?: I FOUND I FLY IN MY DAL TODAY, I WANT A REFUND
```

Providing the output - `print("The content to be printed here")`

```
>>> print("Customer states they have the following problem:\n", customer_reply)
Customer states they have the following problem:
 I FOUND I FLY IN MY DAL TODAY, I WANT A REFUND
```

OUTPUT

The `print()` function

1. Writes to the screen
2. Pass one value, or several separated by commas
3. Every call starts a new line

```
1    print("Hello, Coding Cafe")
2    >>> Hello, Coding Cafe
3
4    print("2 + 2 =", 2 + 2)
5    >>> 2 + 2 = 4
```

INPUT

The `input()` function

1. Stops and waits for the user to type
2. Whatever they type comes back as text
3. Give it a prompt, so they know what you want

```
name = input("What's your name? ")
>>> What's your name? Raghav

print("Hello,", name)
>>> Hello, Raghav
```

PUTTING IT TOGETHER

1. A program is instructions, in order
2. Python reads top to bottom — that is all that happens here

These colors have some significance, we will talk about that soon.

```
1  # greet.py
2  print("Welcome to Coding Cafe")
3
4  name  = input("Your name: ")
5  place = input("Where are you from? ")
6
7  print("Hello,", name)
8  print("Good to see someone from", place)
```

20 AUGUST 2026

fin